
Laboratory 1 – Embedded “Hello World!”

1 Overview and Goals

In this laboratory we will familiarize ourselves with the Texas Instruments (TI) MSP430F5438 Experimenter microcontroller board and IAR Embedded WorkBench. We will start by writing the classic example of a first embedded program, which for a microcontroller is a blinking light.

There are several specific objectives in this lab:

1. Introduce the MSP430F5438 Board
2. Learn to use the IAR Embedded Workbench for code development.
3. Gain familiarity with the circuit diagram for the MSP430F5438 Experimenter board.
4. Interface with peripheral devices using General Purpose Input Output (GPIO).

2 Introduction

Microcontrollers (sometimes abbreviated μC , uC or MCU) are small computers, generally single core on, which have memory, flash and a variety of programmable peripherals. They often have no disk storage, no virtual memory and no operating system. We can view them just like other digital hardware components with the added benefit that we can configure them using software. As a function of this, microcontrollers can be utilized within a wide variety of electronic design projects, to do all sorts of things easily that otherwise would require a large number separate components. Simple microcontrollers are generally very cheap, costing often less than dollar, and hence can be used in almost any project. Of course, it cannot compare to a modern PC in aspects such as speed and processing capability for all but the high performance CPUs, but it is useful in a wide variety of applications where larger processors are not feasible.

Within these laboratories we will be using the TIMSP430f5438 Experimenter board shown in Figure 1. The MSP430F5438 Experimenter Board (MSP-EXP430F5438) is a development platform for the latest generation TI MSP430 MCUs. The MCU comes with a wide variety of peripherals which we will interface within the next 6 laboratories. A brief overview of the peripherals of the Experimenter boards is as follows:

- 100-pin socket for MSP430F5438
- Power Supply sources: USB, FET, 2x AA batteries
- Digital I/O Pins: 34
- Accessible analog inputs (12-bit ADC): 5
- PWM outputs: 12
- Flash Memory (MSP430F5438): 256KB
- RAM (MSP430F5438): 16KB
- Clock Speed (MSP430F5438): 18MHz
- Communication (MSP430F5438):
 - 4x UART/LIN/IrDA/SPI
 - 4x I2C/SPI

- 5-position joystick (up, down, left, right, push down)
- 2 push buttons
- 2 LEDs
- 138x110 grayscale, dot-matrix LCD
- 3-Axis Accelerometer (ADXL330)
- Microphone (Amplified by [TLV2760](#))
- 3.5mm audio output jack (Features [TPA301](#), 350mW Mono Audio Power Amplifier)
- Support for TI Low Power RF Wireless Evaluation Modules and eZ430-RF2500T. Currently supported modules:
 - CC1100/CC1101EMK – Sub-1GHz radio
 - CC2500EMK – 2.4 GHz radio
 - CC2420/CC2430EMK – 2.4 GHz 802.15.4 radio
 - CC2520/CC2530EMK – 2.4 GHz 802.15.4 radio
- USB connectivity for data transfer
- JTAG header for real-time, in-system programming

The MSP430F5438 Experimenter board is shown as follows:

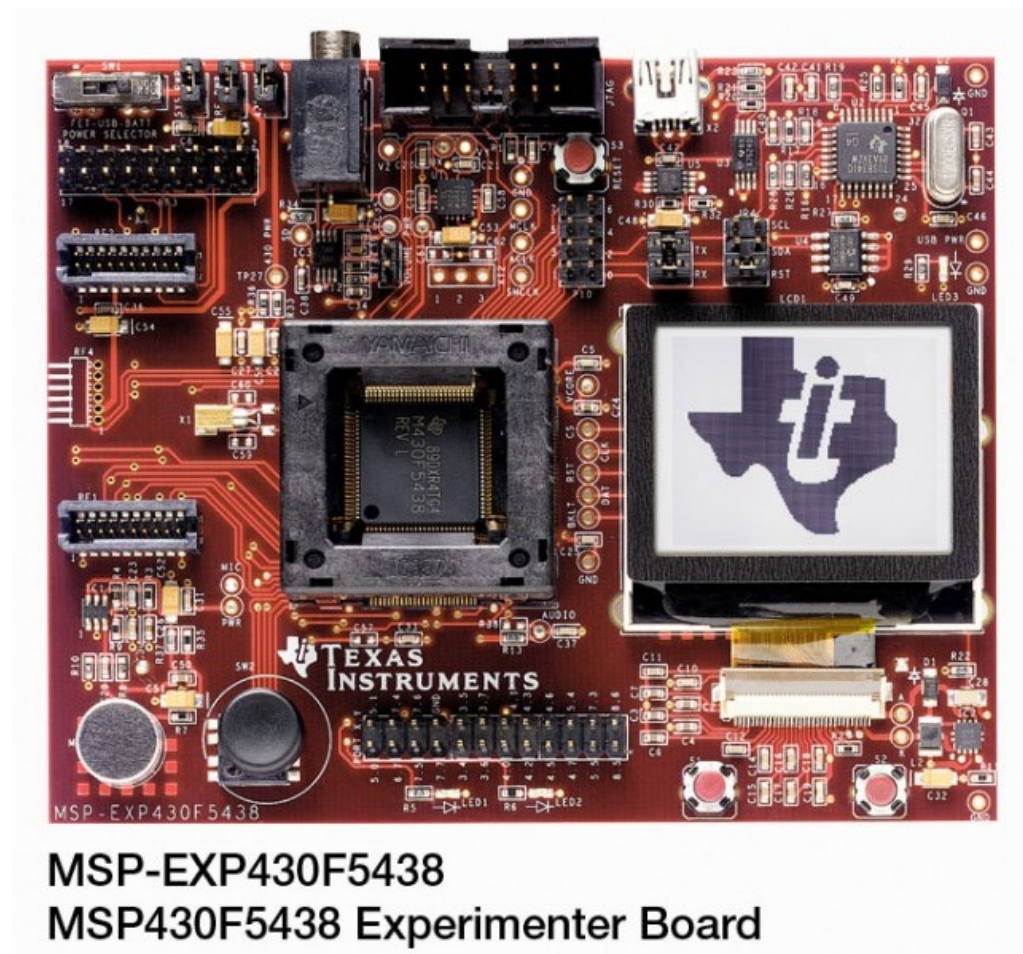


Figure 1. Picture of the microcontroller prototyping board used in this laboratory.

As we will be developing software on the computers within the lab that have to run on the embedded platform (the MSP430F5438 Experimenter), we will require to cross compile the software on the machines in the laboratory, then we will download this code onto the boards. To develop the

object code that we will download to the board we will use Texas Instruments software development platform Code Composer Studio. To interface to the board we will use the MSP-FET430UIF flash emulation tool. This tool has a USB debugging interface which connects a flash-based MSP430 MCU to the PC, allowing real-time, in-system programming and software debugging. The MSP-FET430UIF is shown in Figure 2.



Figure 2. MSP-FET430UIF flash emulation tool

2.1 Useful Documentation

To help new developers get up to speed with the rich set of functionality that is available with the MSP430 there exists extensive documentation, relating to both the MCU and the Experimenter board. The following wiki is populated by TI and is a good place to begin to find documentation and software examples.

http://processors.wiki.ti.com/index.php/MSP-EXP430F5438_Experimenter_Board

The following documents (pdfs) will be required for this and any subsequent laboratories, they are available at the following website and copies are available on GCU Learn.

<http://www.ti.com/tool/msp-exp430f5438>

- MSPF5438 Experimenter User Guide (Rev G.)

<http://www.ti.com/product/msp430f5438a>

- MSP430F543xA, MSP430F541xA Mixed Signal Microcontroller (Rev. B)

Datasheet for the MSP430F5438

<http://www.ti.com/lit/ds/symlink/msp430f5438.pdf>

2.2 Getting started with the MSP430F5438 Experimenter board

After we take the board out of the box, to begin working we require to set JP2 on the board. This allows the board to take power from either the batteries attached, a mini USB connection or the MSP-FET430UIF connection. Out of the box the jumper JP2 will appear as follows:

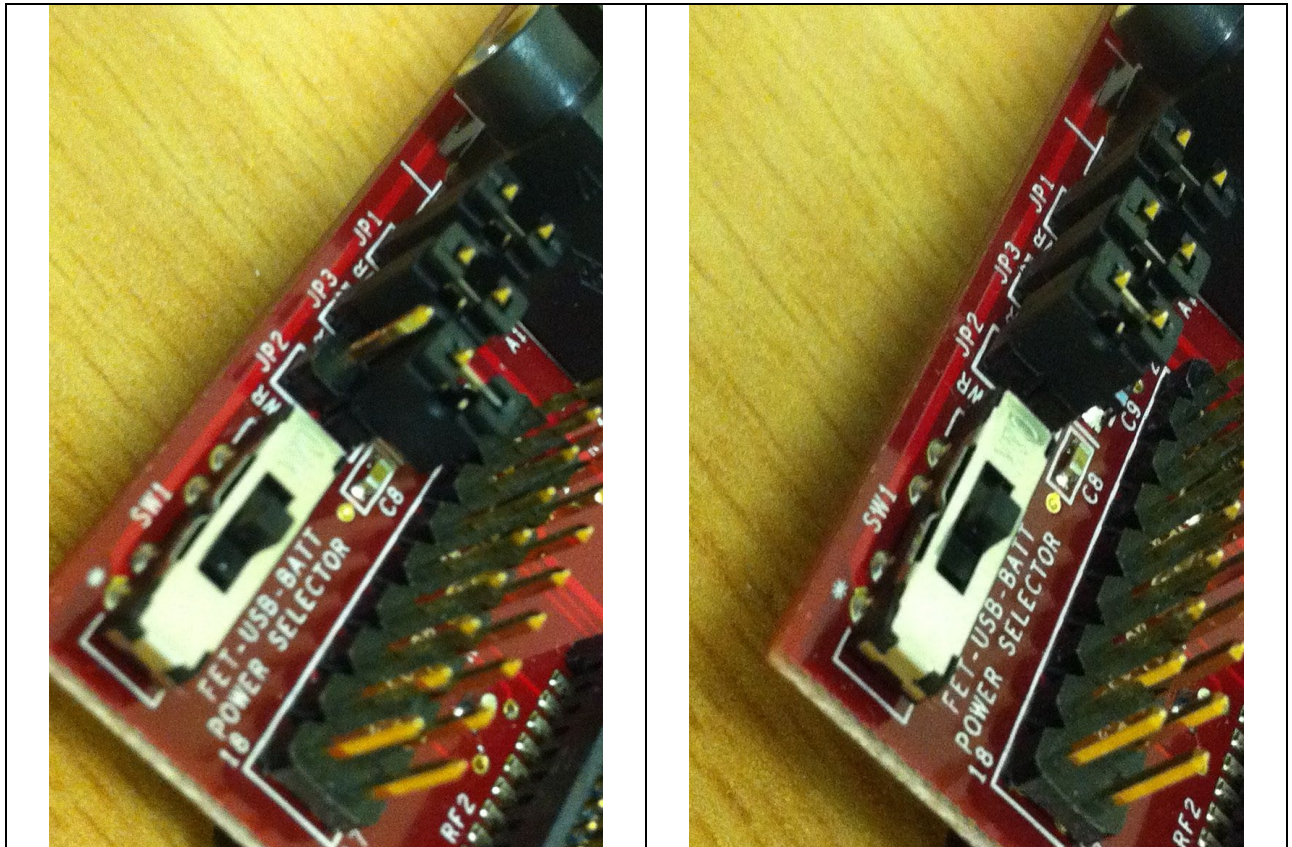


Figure3. Setting the system power jumper for the MSP430F5438 Experimenter

Now to test the board is operating correctly, flip the switch shown above to the battery selection, labelled BATT in Figure 3. The board should now power on and the LCD screen will shown the TI splash screen. Pressing the joystick in the left hand corner will open a menu which will allow you to explore some of the demo software that is given by TI to demonstrate some of the functionality available with the board. This software is called the User Experience and is downloadable from

<http://www.ti.com/tool/msp-exp430f5438>

It is referred to as SLAC227 on this page. Please go to this website and find this software on the page, the link is listed as

- Complete demo software available ([SLAC227](#))

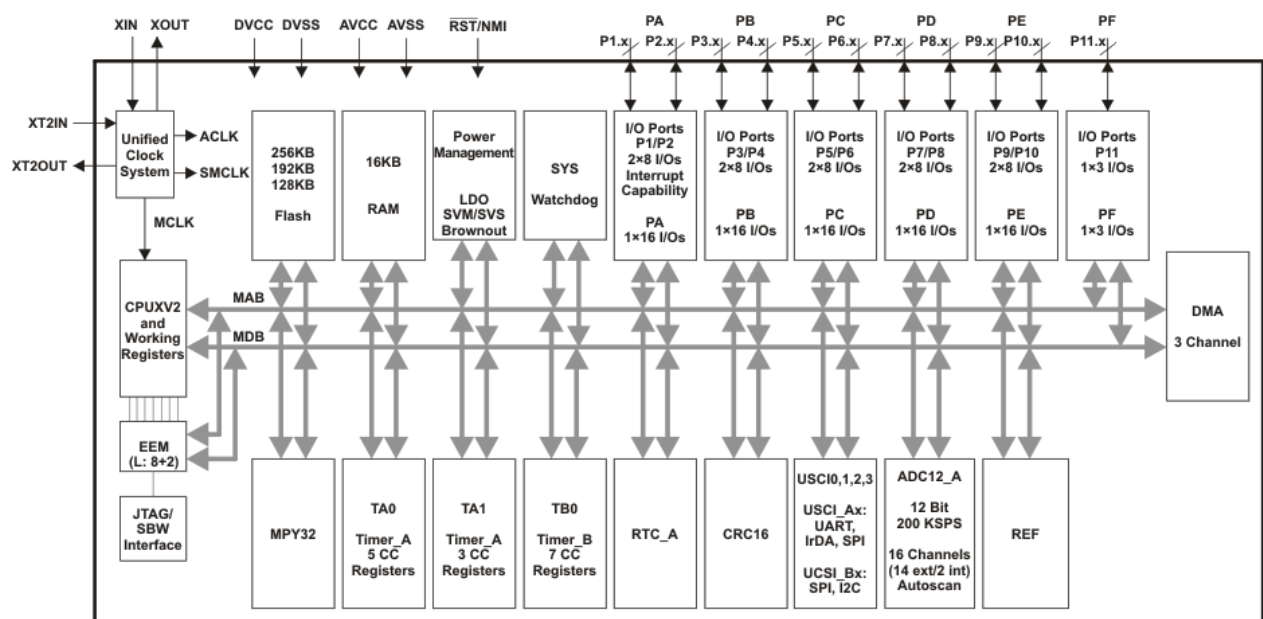
The details of how to install the complete demo software are listed in the MSPF5438 Experimenter User Guide. You should become familiar with this procedure. Details of how to compile and download the software will be given within this laboratory.

3 Digital Output using General Purpose Input Output (GPIO)

In this section of the laboratory we will familiarize ourselves with the microcontroller board, the JTAG programmer and the MSP430 software development tool Code Composer Studio (CCS). We will start by writing the equivalent of the classic example of a first program “Hello World!”, which for the embedded world is a blinking LED.

3.1 Microcontroller peripherals

As had been discussed already the Microcontroller board we are using comes with a rich set of peripherals to allow the device to be utilized in a wide range of circumstances. The functional block diagram for the system is given in Figure 4, which details the specific hardware blocks found within the MSP430F5438.



Note: Supported memory, peripherals, and ports may vary depending upon the device.

Figure4. Functional block diagram MSP430F5438 Microcontroller

Within the laboratories we will utilize a number of the features offered by the MSP430. Within this lab we will be using the General Purpose Input Output (GPIO) ports. These allow us to take inputs from devices such as switches and pushbuttons, and output to LEDs, relays etc. These ports offer a high degree of flexibility as they can be configured via software as both inputs and outputs, which allows the same Microcontroller to be used within multiple designs. It is common for different hardware blocks to share the same output pins, we will see this further in later labs.

The GPIO Ports P1 and P2 are digital I/O buses that can be configured as inputs or outputs. They can be used for simple I/O with switches or LEDs; in later labs we will see much more sophisticated uses of this simple interface. On the MSP430F5438 Experimenter board the LEDs that we can use for output are connected to Port 1. We can see this by looking at the board circuit diagram. This is shown in Figure 5.

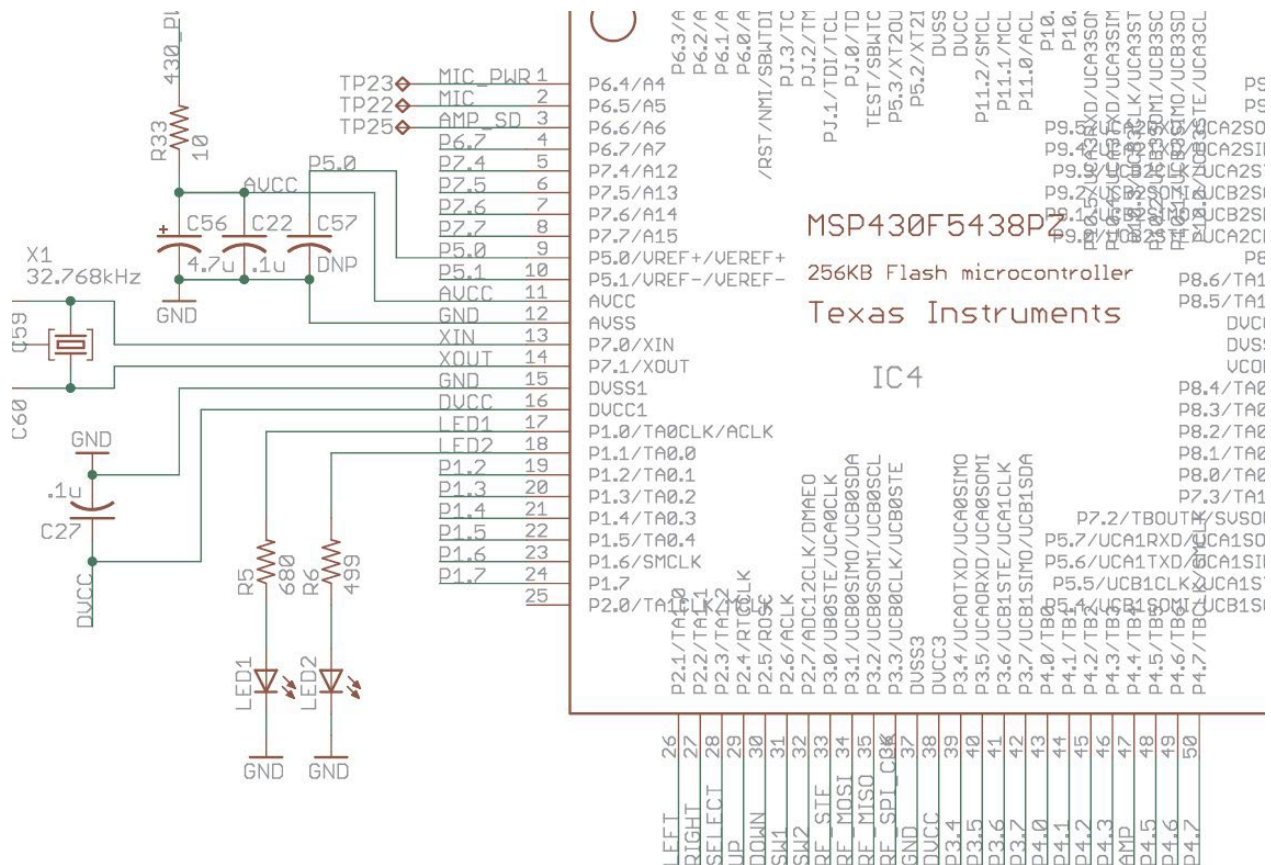


Figure 5. Circuit diagram MSP430F5438 Experimenter board

Lab Question 1. Explain why the resistors are required between the microcontroller output pins and the LEDs?

In order to develop software to control the LEDs, you'll need to know a bit about how ports like P1 work. Each port is a group of 8 pins; these pins' values can be set (if they are outputs) or read (if they are inputs) by the microcontroller. This requires to be configured by the programmer. Each pin's input/output status (i.e. the direction that the pin operates in input or output) can be individually set by manipulating 'registers' in the microcontroller, each register is a group of 8 bits within the microcontroller that may or may not directly affect pins. Once this is set we can then read or write to the pin.

Lab Question 2. What output from the Microcontroller is required to turn on LED1?

At this stage, you need to know about the following registers. Ports other than P1 have similar registers that start with P2, etc:

P1SEL	Set function of each pin in P1. 0=basic digital I/O 1=special function
P1DIR	Set data flow direction of each pin in P1. 0=pin is input 1=pin is output
P1OUT	Set voltage on each output pin in P1. 0=pin low (GND) 1=pin high (Vcc)

To setup the ports to operate as we wish them to we require to manipulate the above registers. The above registers are 8 bits in size, yet we only wish to manipulate one individual bit if we wish to control LED1 in Figure 5. To do this we require to be able manipulate individual bits within a register in software. To do this we can utilize some convenient predefined values BIT0 through BIT7. These are defined in the include file msp4305438.h as

```
#define BIT0          (0x0001)
#define BIT1          (0x0002)
#define BIT2          (0x0004)
#define BIT3          (0x0008)
#define BIT4          (0x0010)
#define BIT5          (0x0020)
#define BIT6          (0x0040)
#define BIT7          (0x0080)
```

If we wish to set pin 0 (bit 0) of port 1 (P1.0) high, we can write

```
P1OUT = BIT0;
```

Yet, this has the problem that if any other pins in P1 had been high before, they would now be low. If we want to set pin 0 but leave the other bits alone, we need to write

```
P1OUT = P1OUT | BIT0;
```

The vertical line (called a 'pipe'). It is a bitwise OR function - that is, it works on one bit at a time. It determines bit 0 of its output by ORing together bit 0 of P1OUT and bit 0 of BIT0, and so on. This will set bit 0 of P1OUT high, but leave the rest of P1OUT unchanged. A C short hand notation is also commonly used.

```
P1OUT |= BIT0;
```

This does the same thing, but uses a shorthand notation to save on typing.

Lab Question 3. From the above we can see how to set an individual bit within a register, what logical operators can we use to clear an individual bit, but leave all others unchanged? Write this in words below, the lab demonstrator will show you how to write this in code.

To toggle the output of a pin, we utilize the bitwise exclusive OR function, as follows:

```
P1OUT ^= BIT0;
```


We will utilize this to blink the LED using the following code

```
for(;;)
{
    P1OUT ^= BIT0;
}
```

Lab Question 4. The above code uses a structure that you would normally avoid in programming, what is it?

3.2 First program – Output to LEDs

Visit IAR User Guide for details regarding creating new project, simulation and debugging options.

1. Paste the following code into the main.c file

```
/**
 * Description: Blink the LED on Port 1.0
 * Author:
 * Data:
 */

#include "io430.h"

void main(void)
{
    // Stop WDT
    WDTCTL = WDTPW + WDTHOLD;

    // Setup P1.0 and an output port
    P1DIR = P1DIR | BIT0;

    // Create main loop
    while(1)
    {
        // Toggle the LED
        P1OUT ^= BIT0;
    }

    return;
}
```

3.3 Debugging

As we have seen, even with the best of intentions sometimes software doesn't function as you would expect. To work out what is going wrong we require a methods of seeing what is

happening within the code. For embedded development we often want to see what is happening at individual register level within the MCU to check that peripherals have been properly configured at startup etc. Code Composer Studio and the MSP-FET430UIF flash emulation tool offer extensive debugging capabilities. We will see how to look at what is happening in the previous code now.

In real time, the LED will be blinking at very high speed hence LED blinking will not be visible to human eye. From this we can conclude the software is operating correctly, we just require to slow it down a bit. We will see that there a number of ways to do this. For just now we will opt for the simplest, a software delay. The following function implements a software delay.

```
void delay(int n)
{
    int i=0;

    // Do nothing for n iterations (spin round in the loop)
    for(i=0;i<n;i++)
    {
    }
    return;
}
```

Lab Question 7. Rewrite the delay function using a different loop structure in C.

3.4 Next laboratory – Input from switches

In the next laboratory we will be exploring how to control the LEDs using the switches. In this laboratory you have seen how to work out what pins the LEDs were connected to and how to setup the ports for those pins as outputs. For the next laboratory complete the following, the information is found in the User Guide for the MSP430F5438.

Lab Question 8. What ports are the push buttons connected to?

Lab Question 9. Write the code to setup these ports pins as an inputs (this is 2 lines).

--
